
Programmation impérative (Novice)

Cahier de travaux pratiques

Licence informatique 2^{ème} année

Université de La Rochelle



Ce document est distribué sous la licence CC-by-nc-nd

<https://creativecommons.org/licenses/by-nc-nd/4.0/deed.fr>

© 2011-2024 Christophe Demko <christophe.demko@univ-lr.fr>



2024-2025_1

Ce cahier de travaux pratiques constitue à notre sens la base des exercices à savoir réaliser pour une pratique basique de la programmation en langage C.

Les lignes commençant par \$ représente une commande *unix*. Le \$ ne fait pas partie de la commande proprement dite mais symbolise l'invite de commande.

Table des matières

1	Prise en main de l'environnement	2
1.1	Utilisation de l'utilitaire <code>make</code>	2
1.2	Utilisation du débogueur <code>gdb</code>	3
1.2.1	Configuration	3
1.2.2	Utilisation	4
1.3	Utilisation de l'utilitaire <code>cmake</code>	6

Liste des figures

Liste des exercices

1	Création d'un exécutable minimal	2
2	Étude des règles par défaut de l'utilitaire <code>make</code>	3
3	Variables pour la compilation C	3
4	Utilisation du débogueur	5
5	Compilation séparée	5
6	<code>CMakeLists.txt</code> minimal	7
7	Création de bibliothèque	9

1 Prise en main de l'environnement

1.1 Utilisation de l'utilitaire make

Sous *unix* (et donc sous *linux* par extension), il existe un utilitaire appelé *make* (prononcer *mec*), permettant de gérer les processus de création de fichiers. C'est un outil indispensable en programmation puisqu'il permet de maintenir une cohérence temporelle entre les fichiers sources et les fichiers compilés ou exécutables.

make est préfiguré avec des dizaines de règles de création, et notamment pour ce qui nous intéresse, avec une règle de création d'un exécutable à partir d'un fichier `.c`.

Il suffit pour cela d'exécuter la commande

```
$ make <executable>
```

et l'utilitaire exécutera une ligne de compilation et d'édition des liens (nous reverrons plus tard ces notions distinctes) permettant de créer `executable` à partir d'`executable.c`



Exercice 1 (*Création d'un exécutable minimal*)

Créez un fichier `exo01.c` contenant le nombre minimum d'instructions pour créer un exécutable. Lancez la commande permettant de créer l'exécutable `exo01`.

Pour lancer l'exécutable créé, il vous suffira de lancer :

```
$ ./exo01
```

Voir *Le piège du répertoire courant* dans [Variable d'environnement](https://fr.wikipedia.org/wiki/Variable_d'environnement)¹.

¹https://fr.wikipedia.org/wiki/Variable_d'environnement



Exercice 2 (Étude des règles par défaut de l'utilitaire make)

En utilisant l'option `-p` de l'utilitaire `make`, générez un fichier `Makefile` dans le dossier temporaire `/tmp` avec la commande suivante :

```
$ make -p > /tmp/Makefile
```

contenant toutes les règles par défaut de l'outil `make` (vous utiliserez à bon escient le manuel de `make`). Éditez ce fichier avec votre éditeur favori (`gedit`). La création de ce fichier n'est là que pour étudier les règles par défaut ; il ne s'agit pas d'un fichier pérenne.

Lors de l'exécution de cette commande, l'utilitaire `make` vous affichera probablement un message d'erreur signifiant que la commande a été lancée sans cible spécifique et qu'il n'y a pas de cible par défaut (définie dans un fichier `Makefile`). Néanmoins, la création du fichier `/tmp/Makefile` a bien été effectuée.

Les variables intéressantes à configurer pour la programmation C sont au nombre de 6 :

CPPFLAGS options du préprocesseur C (`PreProcessorFLAGS`).

CFLAGS options du langage C.

LDFLAGS options de l'aggrégateur de liens (*linker* en anglais). À l'origine, *ld* était un acronyme pour *load*).

LDLIBS options de l'aggrégateur de liens ne contenant que les bibliothèques à lier (`LoadLIBraries`)

LOADLIBES options de l'aggrégateur de liens ne contenant que les répertoires des bibliothèques à lier

TARGET_ARCH le processeur cible de la compilation ; il est en effet possible de compiler sous *unix* pour une exécution prévue sous *windows* ou sous *macos*



Exercice 3 (Variables pour la compilation C)

1. Quelles sont les règles permettant de créer un exécutable à partir d'un fichier écrit en C ? Comment sont utilisées les variables précédemment citées ?
2. Quelles sont les règles permettant de créer un fichier objet à partir d'un fichier écrit en C ? Comment sont utilisées les variables précédemment citées ?

1.2 Utilisation du débogueur gdb

1.2.1 Configuration

Un débogueur, ou *dévermineur* pour être précis, est un outil permettant d'observer le déroulement d'un programme. C'est l'*outil* indispensable du *bon* programmeur, en opposition avec celui qui préfère ajouter à son code des impressions de valeur intermédiaires pour les effacer lorsque le programme est finalisé.

Pour utiliser le débogueur (`gdb` dans le cas présent), il suffit de compiler et d'éditer les liens avec l'option `-g`. `make` a prévu un mécanisme pour le faire naturellement. Si un fichier nommé `Makefile`

ou `makefile` est présent dans le répertoire courant, `make` s'en sert comme entrée pour les règles de création. Pour ce qui nous concerne, il suffit donc de créer un `Makefile` contenant deux lignes

```
CFLAGS=-g
LDFLAGS=-g
```

`CFLAGS=-g` indique au compilateur l'ensemble des options (CFLAGS) qu'il devra prendre en compte (ici `-g`).

`LDFLAGS=-g` indique à l'éditeur des liens l'ensemble des options (LDFLAGS) qu'il devra prendre en compte (ici `-g` également). Chaque exécutable sera alors créé pour pouvoir être exécuté avec le débogueur. Il suffit pour cela d'exécuter la commande

```
$ gdb <executable>
```

1.2.2 Utilisation

Plusieurs commandes permettent d'utiliser `gdb`. Tout au long des TP, nous les verrons en détail.

Voici les principales à connaître :

help menu général de l'aide ;

list permet de visualiser le code source du programme ;

run permet de lancer l'exécution du programme ;

jump numéro permet de reprendre l'exécution du programme à la ligne numéro ;

until numéro permet de continuer le programme jusqu'à la ligne numéro ;

step ou next permet d'exécuter la ligne suivante (nous verrons par la suite la différence entre ces deux commandes) ;

finish termine l'exécution ;

print exp permet de connaître la valeur de l'expression `exp` ;

set var variable_name=exp permet de modifier la valeur de la variable `variable_name` et de lui assigner la valeur `exp` ;

display exp permet d'imprimer la valeur de l'expression `exp` à chaque fois que le programme s'arrête ;

undisplay num retire de la liste des expressions à afficher l'expression de numéro `num` ;

x &var permet d'examiner la mémoire de la variable `var` avec différents formats ;

break numéro permet de faire arrêter l'exécution du programme à la ligne numéro ;

delete numéro permet de effacer le `break` numéro (attention, ce n'est pas le numéro de ligne mais le numéro du `break`).

Il faut noter que la plupart de ces commandes sont utilisables avec des abréviations (`l` pour `list`, etc).



Exercice 4 (Utilisation du débogueur)

1. Écrire le programme suivant dans un fichier `gcd.c`

```
#include <stdlib.h>

int main(void) {
    int a, b;
    a %= b;
    b %= a;
    return EXIT_SUCCESS;
}
```

2. Compiler le programme pour pouvoir l'utiliser avec le débogueur ;
 3. Lancer le programme avec le débogueur ;
 4. Dans le débogueur, affecter à la variable `a` la valeur 32 et à la variable `b` la valeur 56 ;
 5. À l'aide des commandes du débogueur, exécuter la ligne `a %= b ;`
 - si `a` est plus grand que `b`
- et la ligne `b %= a ;`
- si `b` est plus grand que `a` jusque l'un d'entre eux soit nul.



Exercice 5 (Compilation séparée)

La fonction *Plus Grand Commun Diviseur* (*GreatestCommonDivisor*) peut être intéressante à définir dans un fichier séparé de la fonction principale `main`. Pour permettre une extension aisée des fonctions arithmétiques, nous l'écrirons dans un fichier `arithmetic.c` et nous la nommerons `arithmetic_gcd`.

1. Écrire dans un fichier `arithmetic.c` une fonction `arithmetic_gcd` permettant de calculer le *pgcd* de deux nombres. Le code sera contenu dans une fonction dont la déclaration est :

```
extern unsigned int arithmetic_gcd(unsigned int a, unsigned int b);
```

2. Écrire un fichier `gcd-32-56.c` permettant d'afficher le *pgcd* de 32 et 56 :

```
#include <stdio.h>
#include <stdlib.h>

extern unsigned int arithmetic_gcd(unsigned int a, unsigned int b);

int main(void) {
    int a = 32;
    int b = 56;
    printf("%u\n", arithmetic_gcd(a, b));
    return EXIT_SUCCESS;
}
```

```
}
```

3. Écrire dans le `Makefile` les instructions pour créer l'exécutable `gcd-32-56` à partir de `gcd-32-56.o` et `arithmetic.o`:

```
CFLAGS=-g
LDFLAGS=-g
```

```
all: gcd-32-56
```

```
gcd-32-56: gcd-32-56.o arithmetic.o
```

Testez le résultat en exécutant directement

```
$ make
```

La cible `all` est exécuté lorsqu'aucun argument n'est passé à la commande `make`. La cible `all` dépend de `gcd-32-56` qui est donc créé après création des `.o` qui dépendent par défaut de leurs `.c` respectifs.

4. Écrire un fichier `arithmetic.h` permettant de remplacer la ligne

```
extern unsigned int arithmetic_gcd(unsigned int a, unsigned int b);
```

par une directive d'inclusion du préprocesseur. Testez le résultat.

1.3 Utilisation de l'utilitaire `cmake`

`cmake`² est un utilitaire multi-plateforme permettant, à l'aide d'un langage de définition de projets, de définir comment et quels sont les fichiers à créer dans un projet de programmation. Il fonctionne aussi bien sous *window*, *unix*, ou *macos* et permet de générer des projets prêts à être utilisés avec :

- l'utilitaire `make`
- l'environnement de développement pour *windows* [Visual Studio](https://visualstudio.com/)³
- l'environnement de développement pour *windows*, *unix*, ou *macos* [CodeBlocks](http://www.codeblocks.org/)⁴
- l'environnement de développement pour *windows*, *unix*, ou *macos* [CodeLite](http://codelite.org/)⁵
- l'environnement de développement pour *windows*, *unix*, ou *macos* [Eclipse](https://eclipse.org/)⁶
- l'environnement de développement pour *windows*, *unix*, ou *macos* [Sublime Text](http://www.sublimetext.com/)⁷
- l'environnement de développement pour *unix* ou *macos* [KDevelop](https://www.kdevelop.org/)⁸
- l'environnement de développement pour *unix* ou *macos* [Kate](http://kate-editor.org/)⁹
- l'environnement de développement pour *macos* [XCode](https://developer.apple.com/xcode/)¹⁰

²<https://cmake.org/>

³<https://www.visualstudio.com/>

⁴<http://www.codeblocks.org/>

⁵<http://codelite.org/>

⁶<https://eclipse.org/>

⁷<http://www.sublimetext.com/>

⁸<https://www.kdevelop.org/>

⁹<http://kate-editor.org/>

¹⁰<https://developer.apple.com/xcode/>

La définition du projet se fait dans un fichier nommé `CMakeLists.txt`.



Exercice 6 (*CMakeLists.txt minimal*)

Créez un dossier `src/` et copiez y les fichiers `gcd-32-56.c`, `arithmetic.c` et `arithmetic.h` précédemment créés. Avec votre éditeur de texte favori, créez dans ce dossier `src/` un fichier `CMakeLists.txt` contenant :

```
cmake_minimum_required(VERSION 3.0)
project(Arithmetic C)
add_executable(gcd-32-56 gcd-32-56.c arithmetic.c arithmetic.h)
```

La ligne `add_executable(gcd-32-56 gcd-32-56.c arithmetic.c arithmetic.h)` indique à `cmake` qu'il faudra créer un exécutable nommé `gcd-32-56` à partir des fichiers sources `gcd-32-56.c` et `arithmetic.c` (`arithmetic.h` n'est pas utilisé pour l'aggrégation : par convention, `cmake` n'utilise que les dates de modification des fichiers d'entête pour déterminer si l'aggrégation doit être relancé). `cmake` reconnaît le type des fichiers d'après leur extension : `gcd-32-56.c` et `arithmetic.c` seront compilés pour créer leurs `.o` respectifs et aggrégés pour créer l'exécutable `gcd-32-56`.

Créez un dossier `build/` au même niveau que `src/` :

```
$ mkdir build
$ tree
.
+-- build
+-- src
    +-- arithmetic.c
    +-- arithmetic.h
    +-- CMakeLists.txt
    +-- gcd-32-56.c
```

2 directories, 4 files

Placez vous dans le dossier `build/` et exécutez la ligne de commande `cmake ../src/` :

```
$ cd build/
$ cmake ../src/
-- The C compiler identification is GNU 5.2.1
-- The CXX compiler identification is GNU 5.2.1
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
```

```
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: .../build
```

À ce stade, le dossier `build/` est prêt pour la compilation et l'édition des liens. Tout un aréopage de fichiers ont été créés par l'utilitaire `cmake`.

```
$ tree
.
+-- CMakeCache.txt
+-- CMakeFiles
|   +-- 3.2.2
|   |   +-- CMakeCCompiler.cmake
|   |   +-- CMakeCXXCompiler.cmake
|   |   +-- CMakeDetermineCompilerABI_C.bin
|   |   +-- CMakeDetermineCompilerABI_CXX.bin
|   |   +-- CMakeSystem.cmake
|   |   +-- CompilerIdC
|   |   |   +-- a.out
|   |   |   +-- CMakeCCompilerId.c
|   |   +-- CompilerIdCXX
|   |   |   +-- a.out
|   |   |   +-- CMakeCXXCompilerId.cpp
|   +-- cmake.check_cache
|   +-- CMakeDirectoryInformation.cmake
|   +-- CMakeOutput.log
|   +-- CMakeTmp
|   +-- feature_tests.bin
|   +-- feature_tests.c
|   +-- feature_tests.cxx
|   +-- Makefile2
|   +-- Makefile.cmake
|   +-- gcd-32-56.dir
|   |   +-- build.make
|   |   +-- cmake_clean.cmake
|   |   +-- DependInfo.cmake
|   |   +-- depend.make
|   |   +-- flags.make
|   |   +-- link.txt
|   |   +-- progress.make
```

```
|   +-- progress.marks
|   +-- TargetDirectories.txt
+-- cmake_install.cmake
+-- Makefile
```

6 directories, 29 files

Il n'y a plus qu'à exécuter make dans le dossier build/ :

```
$ make
Scanning dependencies of target gcd-32-56
[ 50%] Building C object CMakeFiles/gcd-32-56.dir/gcd-32-56.c.o
[100%] Building C object CMakeFiles/gcd-32-56.dir/arithmetic.c.o
Linking C executable gcd-32-56
[100%] Built target gcd-32-56
```

L'exécutable gcd-32-56 a été créé ; il n'y a plus qu'à l'exécuter.

```
$ ./gcd-32-56
8
```

Il est possible de demander à la commande make d'afficher les commandes exécutées en ajoutant VERBOSE=1 :

```
$ make VERBOSE=1
```

La commande make accepte des arguments supplémentaires. Pour les connaître :

```
$ make help
```



Exercice 7 (Création de bibliothèque)

Pour créer une bibliothèque, il faut indiquer à cmake les fichiers sources dont elle dépend :

```
cmake_minimum_required(VERSION 3.0)

project(Arithmetic C)

add_executable(gcd-32-56 gcd-32-56.c arithmetic.c arithmetic.h)
add_library(arithmetic SHARED arithmetic.c arithmetic.h)
```

La ligne **add_library**(arithmetic SHARED arithmetic.c arithmetic.h) indique à cmake qu'il faudra créer une librairie partagée nommée arithmetic à partir du fichier arithmetic.c.

Relancez la compilation et vérifiez que la librairie est bien créée dans le dossier build/.